

Traffic Collision Data Analysis

Analyze large-scale traffic collision datasets using PySpark and AWS services. Apply data analytics techniques to clean, transform, and explore crash data, drawing meaningful insights to support traffic safety and urban planning.

1. Data Preparation –

Write code to load the data and check the schema

Data set is stored in Drive. Data is read using SparkSession

Top 5 entries in Collisions data is displayed.

Column names and their values are validated

```
# Write code to load the data and check the schema
```

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName('Basics').getOrCreate()
collisions_df = spark.read.csv('/content/drive/MyDrive/C3-Cloud_Computing_and_big_data/project_Traffic_Collision/sample_collisions.csv', header=True, inferSchema=True)
collisions_df.show(5)
```

case_id	jurisdiction	officer_id	reporting_district	chp_shift	population	county_city_location	county_location	special_condition	beat_type
8000993.0	1942.0	39335	1676	not chp	>250000	1942	los angeles	0.0	not
4232117.0	3008.0	1284	22	not chp	100000 to 250000	3008	orange	0.0	not
9.1389627E7	9285.0	019223	NULL	1400 thru 2159	unincorporated	5800	yuba	0.0	chp county road
3589817.0	3610.0	50586	3610	not chp	100000 to 250000	3610	san bernardino	0.0	not
9111695.0	1900.0	649318	0261	not chp	25000 to 50000	1917	los angeles	0.0	not

only showing top 5 rows

2. Data Cleaning

```
#Display Schema & Sample Data
collisions_df.printSchema()
```

... Show hidden output

```
collisions_df.count()
```

935791

As whole data set can be loaded and read, no sampling technique is used

2.1 Missing Values –

```
#Check for Missing Values
```

```
from pyspark.sql import functions as F

missing_df = collisions_df.select(
    [F.count(F.when(F.col(c).isNull(), c)).alias(c) for c in collisions_df.columns]
)

missing_df.show(truncate=False)
```

case_id	jurisdiction	officer_id	reporting_district	chp_shift	population	county_city_location	county_location	special_condition	beat_type	chp_beat_type	chp_beat
0	1158	2264	552653	0	165	0	0	51730	0	0	274

Find the percentage of missing values to identify the Sparse Columns

Sparse Column – Column that contains high percentage of missing values -

```
#Drop Sparse Columns
# a sparse column is a column that contains a very high proportion of missing values (NULLs) compared to the total number of rows.

total_rows = collisions_df.count()

missing_values_percentage = (
    collisions_df.select([
        F.round((F.count(F.when(F.col(c).isNull(), c)) / total_rows * 100),2)
        .alias(c) for c in collisions_df.columns
    ])
)

missing_values_percentage.show(truncate=False)
```

case_id	jurisdiction	officer_id	reporting_district	chp_shift	population	county_city_location	county_location	special_condition	beat_type	chp_beat_type	chp_beat
0.0	0.12	0.24	59.06	0.0	0.02	0.0	0.0	5.53	0.0	0.0	0.03

Below fields were dropped due to high sparsity and limited relevance to the current analysis scope -

caltrans_county - 72.89%

location_type - 72.89

side_of_highway - 72.89

Latitude - 71.5

Longitude - 71.5

```
#Missing Data percentage is more than 70%, hence removing the columns
collisions_df_clean = collisions_df.drop("caltrans_county", "location_type", "side_of_highway", "Latitude", "Longitude")

revised_col_count = len(collisions_df_clean.columns)
revised_col_count
```

60

Convert Datatypes -

#Convert Data Types

```
from pyspark.sql.types import IntegerType, LongType, DoubleType
```

```
df_clean_typed = (  
    collisions_df_clean
```

```
    #Ids
```

```
    .withColumn("case_id", F.col("case_id").cast("String"))  
    .withColumn("jurisdiction", F.col("jurisdiction").cast(IntegerType()))
```

```
    #Count fields
```

```
    .withColumn("killed_victims", F.col("killed_victims").cast(IntegerType()))  
    .withColumn("injured_victims", F.col("injured_victims").cast(IntegerType()))  
    .withColumn("party_count", F.col("party_count").cast(IntegerType()))  
    .withColumn("pcf_violation", F.col("pcf_violation").cast(IntegerType()))
```

```
    #Flag Fields
```

```
    .withColumn("intersection", F.col("intersection").cast(IntegerType()))  
    .withColumn("state_highway_indicator", F.col("state_highway_indicator").cast(IntegerType()))  
    .withColumn("tow_away", F.col("tow_away").cast(IntegerType()))  
    .withColumn("not_private_property", F.col("not_private_property").cast(IntegerType()))
```

```
)
```

#Verify the schema after converting datatypes

```
df_clean_typed.printSchema()
```

Handle missing Values –

Missing values in Categorical fields are replaced by “Unknown”

Missing Values in Numerical Fields are replaced by “0”

Missing values in Binary Flag Columns are replaced by “0”

Missing values in “pcf_violation” column which represents code is replaced by “-1” which is a place holder for missing values

Missing values in Location/Administrative Numeric Codes are replaced by “-1”

Missing Values in “Population” field of String Data type is replaced by “Unknown”

```
#Handle Missing Values
```

```
final_df = df_clean_typed.fillna({
  #Categorical
  "control_device": "Unknown",
  "lighting": "Unknown",
  "motor_vehicle_involved_with": "Unknown",
  "pedestrian_action": "Unknown",
  "road_surface": "Unknown",
  "road_condition_1": "Unknown",
  "type_of_collision": "Unknown",
  "weather_1": "Unknown",
  "primary_collision_factor": "Unknown",
  "pcf_violation_category": "Unknown",
  "officer_id": "Unknown_Officer",
  "statewide_vehicle_type_at_fault": "Unknown",
  "chp_vehicle_type_at_fault": "Unknown",
  "pcf_violation_subsection": "Unknown",
  "direction": "Unknown",
  "beat_number": "Unknown",
  "reporting_district": "Unknown",

})

final_df = final_df.fillna({
  #Numeric
  "injured_victims": 0,
  "killed_victims": 0,
  "party_count": 0,
  #Flags
  "intersection": 0,
  "tow_away": 0,
  "state_highway_indicator": 0
})

#pcf Violation
final_df = final_df.withColumn(
  "pcf_violation",
  F.when(F.col("pcf_violation").isNull(), -1)
    .otherwise(F.col("pcf_violation"))
)

final_df = (
  final_df
    .withColumn("caltrans_district",
      F.coalesce(F.col("caltrans_district"), F.lit(-1)))
    .withColumn("state_route",
      F.coalesce(F.col("state_route"), F.lit(-1)))
    .withColumn("jurisdiction",
      F.coalesce(F.col("jurisdiction"), F.lit(-1)))
    .withColumn("postmile",
      F.coalesce(F.col("postmile"), F.lit(-1.0)))
    .withColumn("special_condition",
      F.coalesce(F.col("special_condition"), F.lit(-1.0)))
)

#Population being a String
final_df = final_df.fillna({"population": "Unknown"})
```

Removed all the null values in below columns as null percentage is very less -
chp_beat_class, primary_road, secondary_road

```
final_df = final_df.filter(  
    F.col("chp_beat_class").isNotNull() &  
    F.col("primary_road").isNotNull() &  
    F.col("secondary_road").isNotNull()  
)
```

Validate the missing percentage after handling the missing values to see if it is 0

```
final_df.select([  
    F.count(F.when(F.col(c).isNull(), c)).alias(c)  
    for c in final_df.columns  
]).show(truncate=False)
```

case_id	jurisdiction	officer_id	reporting_district	chp_shift	population	county_city_location	county_location	special_condition	beat_type
0	0	0	0	0	0	0	0	0	0

2.2 Fixing Columns

As "Case ID" uniquely identifies each row, if there are same Case IDs then rows are duplicate.
Hence removing the Duplicate rows by validating if 2 columns have same Case IDs

```
#Remove Duplicates  
print("Number of duplicate rows : ", final_df.groupBy("case_id").count().filter("count > 1").count())  
  
#There are 27 duplicates  
df_dup = final_df.count()  
print("Number of rows before removing duplicates : ", df_dup)  
  
#Dedup  
final_df = final_df.dropDuplicates(["case_id"])  
print("Number of rows after removing duplicates : ", final_df.count())
```

```
Number of duplicate rows : 27  
Number of rows before removing duplicates : 935505  
Number of rows after removing duplicates : 935478
```

Detect Outliers using IQR –

Outliers are checked for numeric Columns –

Columns "pedestrian_killed_count", "motorcyclist_killed_count", "killed_victims",
"bicyclist_killed_count" have min, q1, q2, q3 and max as 0 hence no outliers were found

```

# Detect Outliers using IQR

# List of numerical columns to check for outliers

numeric_cols = [

    "distance",
    "party_count",

    # -- Injured Count
    "injured_victims",
    "severe_injury_count",
    "other_visible_injury_count",
    "complaint_of_pain_injury_count",
    "pedestrian_injured_count",
    "bicyclist_injured_count",
    "motorcyclist_injured_count"

    # -- Fatality Count
    "pedestrian_killed_count",
    "motorcyclist_killed_count",
    "killed_victims",
    "bicyclist_killed_count",

]

```

```
final_df.select(numeric_cols).summary().show()
```

summary	distance	party_count	injured_victims	severe_injury_count	other_visible_injury_count	complaint_of_pain_injury_count
count	935791	935791	935791	935791	935791	935791
mean	679.5426970765903	1.981310997861702	0.5583875031924863	0.025971611182411456	0.14916151149134796	0.38325438051872696
stddev	12959.761507998857	0.6763481407934031	0.8736196797579023	0.1762601459952524	0.42267584352452137	0.7393070575915799
min	0.0	1	0	0	0	0
25%	0.0	2	0	0	0	0
50%	100.0	2	0	0	0	0
75%	488.0	2	1	0	0	1
max	8363520.0	20	27	8	16	22

To find the number of outliers in each of the above numeric columns, a new binary outlier flag column called "{col_name}_outlier" is added.

```

def detect_iqr_outliers(df, col_name, multiplier=1.5):
    """
    Adds a binary outlier flag column using IQR method.
    """

    # Drop nulls for quantile calculation
    #non_null_df = df.filter(F.col(col_name).isNotNull())

    # Compute Q1 and Q3
    q1, q3 = final_df.approxQuantile(col_name, [0.25, 0.75], 0.01)
    iqr = q3 - q1

    lower_bound = q1 - multiplier * iqr
    upper_bound = q3 + multiplier * iqr

    return df.withColumn(
        f"{col_name}_outlier",
        F.when(
            (F.col(col_name) < lower_bound) |
            (F.col(col_name) > upper_bound),
            1
        ).otherwise(0)
    )

```

```
df_outliers = final_df

for col in numeric_cols:
    df_outliers = detect_iqr_outliers(df_outliers, col)

#View Outliers Count

for col in numeric_cols:
    df_outliers.groupBy(f"{col}_outlier").count().show()
```

2.3 Outlier Analysis

1. Distance is the only valid Outlier.

Instead of removing, capping all the values above 100,000 with 100,000 is used

Distance exhibited **extreme right skew** with large maximum values.

Instead of removing entire collision records, distance values were capped using a high-percentile threshold to mitigate the influence of data errors while preserving all collision events for analysis.

1. Outliers in injured related columns represent **rare but valid severe events**.

Changing these values would **destroy important information**

```
count_gt_100000 = final_df.filter(F.col("distance") > 100000)
print("Count of rows having distance value > 100000 : ", count_gt_100000.count())

final_df = final_df.withColumn(
    "distance_capped",
    F.when(F.col("distance") > 100000, 100000).otherwise(F.col("distance"))
)

final_df.select("distance_capped").summary().show()
```

Count of rows having distance value > 100000 : 80

```
+-----+-----+
|summary| distance_capped|
+-----+-----+
| count|          935791|
| mean| 637.3804450352694|
| stddev| 2443.2486741426087|
| min|           0.0|
| 25%|           0.0|
| 50%|          100.0|
| 75%|          488.0|
| max|        100000.0|
+-----+-----+
```

3. Exploratory Data Analysis

3.1.1 Data Preparation

Classify Variables into Categorical and Numerical

Categorical -

control_device, lighting, motor_vehicle_involved_with, pedestrian_action, road_surface, road_condition_1, type_of_collision, weather_1, primary_collision_factor, pcf_violation_category, officer_id, statewide_vehicle_type_at_fault, chp_vehicle_type_at_fault, pcf_violation_subsection, direction, beat_number, reporting_district

Numeric –

other_visible_injury_count, complaint_of_pain_injury_count, pedestrian_killed_count, pedestrian_injured_count, bicyclist_killed_count, bicyclist_injured_count, motorcyclist_killed_count, motorcyclist_injured_count, severe_injury_count, killed_victims injured_victims, party_count

Encode Categorical Values –

Categorical variables must be numerically encoded before modeling or advanced analytics

Categorical variables are encoded using StringIndexer

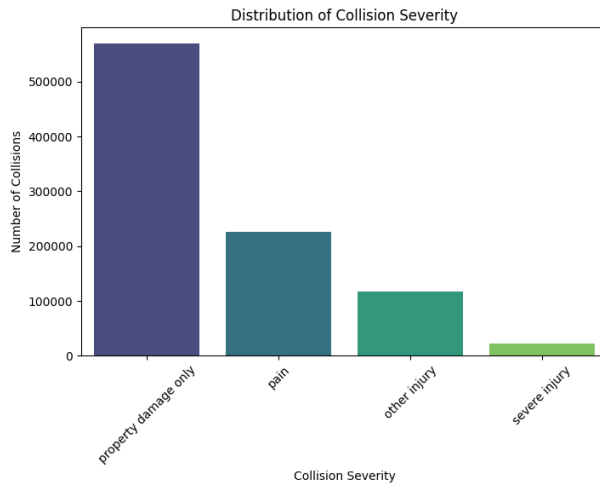
```
from pyspark.ml.feature import StringIndexer

categorical_cols = [
    "primary_collision_factor",
    "pcf_violation",
    "weather_1",
    "lighting",
    "road_surface",
    #"intersection",
    "collision_severity",
]

for col in categorical_cols:
    if col in final_df.columns:
        indexer = StringIndexer(
            inputCol=col,
            outputCol=f"{col}_index",
            handleInvalid="keep"
        )
        final_df = indexer.fit(final_df).transform(final_df)
    else:
        print(f"Column not found, skipping: {col}")
```


3.1.2 Analyze the distribution of Collision Severity

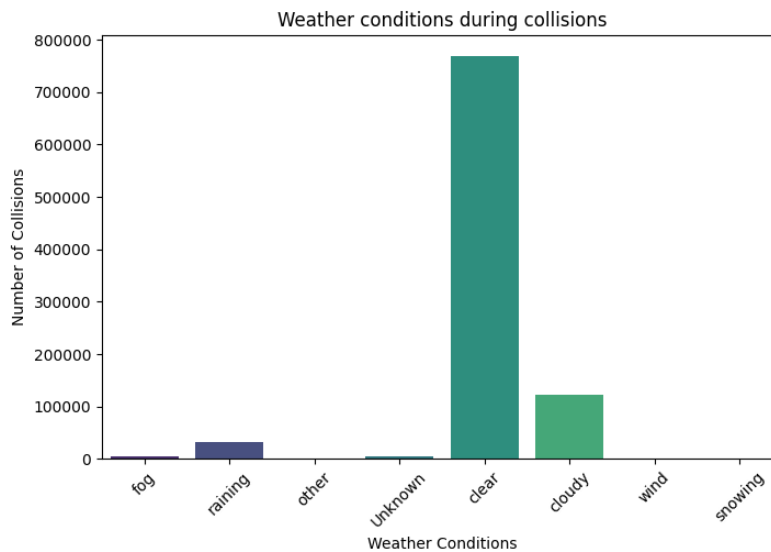
Most collisions fall under lower-severity categories, while severe collisions occur far less frequently. As in real life, high-severity accidents are rare but critically important.



3.1.3 Weather conditions during Collisions

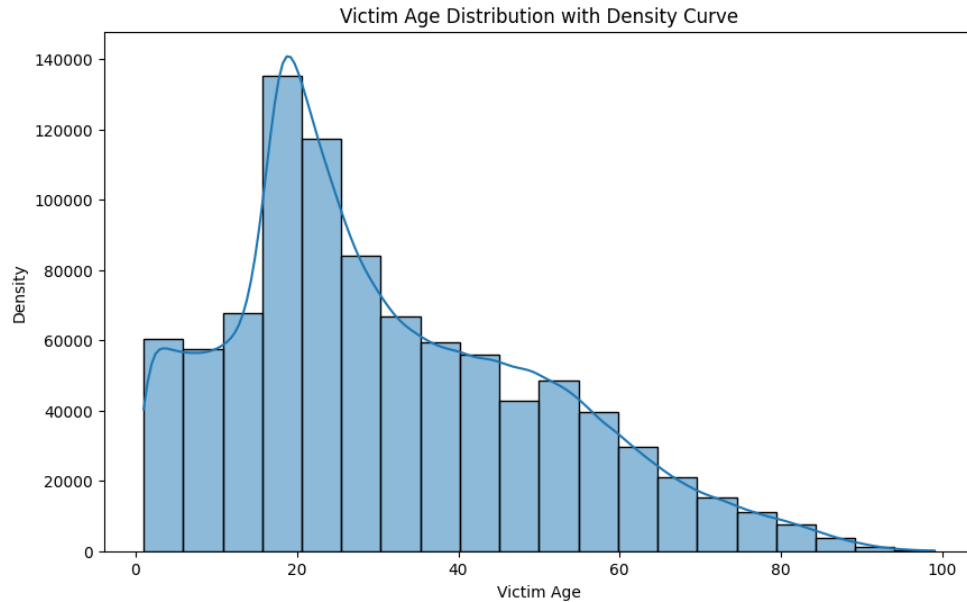
Most collisions occur during clear weather conditions, followed by cloudy and rainy conditions. This trend is expected due to higher traffic volume in normal weather.

Adverse weather conditions such as fog, snow, and wind account for a smaller share of collisions, likely due to reduced exposure and lower driving frequency.

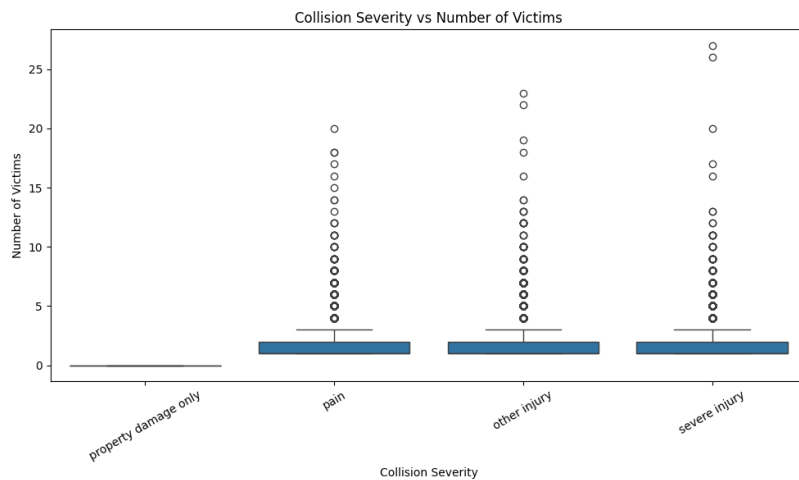


3.1.4 Victim Age Distribution

The victim age distribution is positively skewed with a clear peak in the late-teen to early-adult range(18-25 years), indicating that most collision victims are younger adults.



3.1.5 Collision Severity vs Number of Victims

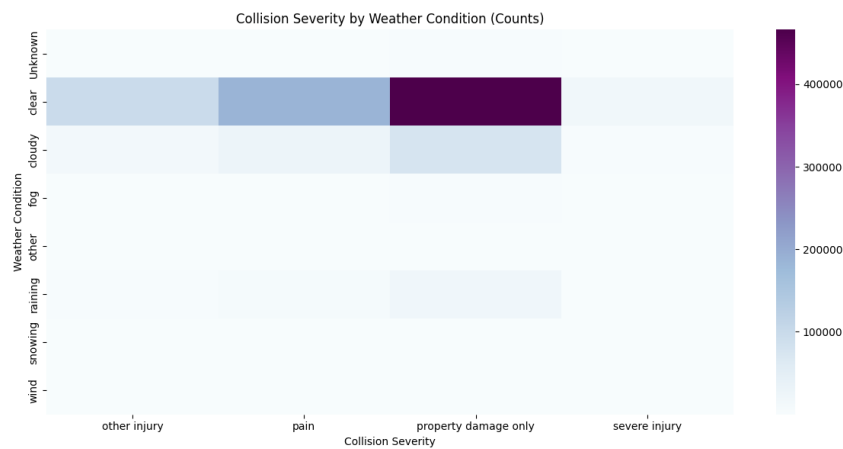


- **Property damage only** collisions are non-injury events and clearly separated from other severity categories.
- **Pain-level** collisions usually involve a small number of victims, there are **rare multi-victim incidents**
- **Other injury** collisions show **greater spread and more extreme victim counts**, suggesting higher impact per event.
- **Severe injury collisions** are associated with **greater variability and the highest-risk, high-impact events**, even though the typical collision still involves a small number of victims.

3.1.6 Weather Conditions vs Collision Severity

collision_severity	other injury	pain	property damage only	\
weather_1				
Unknown	560.0	1193.0	2794.0	
clear	97808.0	187202.0	466004.0	
cloudy	14149.0	28847.0	76527.0	
fog	517.0	754.0	2490.0	
other	82.0	178.0	797.0	
raining	3336.0	7961.0	20419.0	
snowing	148.0	220.0	981.0	
wind	54.0	89.0	233.0	

collision_severity	severe injury
weather_1	
Unknown	77.0
clear	18666.0
cloudy	2572.0
fog	140.0
other	24.0
raining	621.0
snowing	21.0
wind	14.0



- The heatmap shows that most collisions occur under the most frequent weather categories (typically clear/normal conditions), as they provide high driving exposure.
- Property damage only is the most frequent severity across weather
- Cloudy followed by raining is next contributor after clear

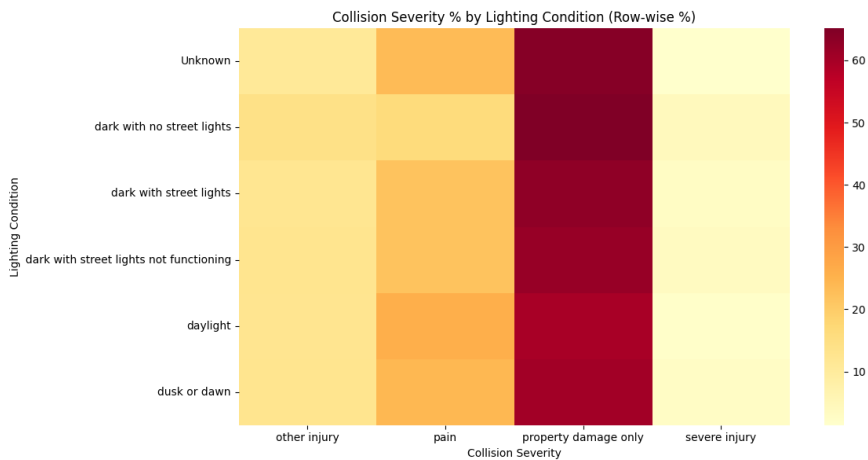
3.1.7 Lighting Conditions Vs Collision Severity

Count of collisions for given lighting Condition and Collision Severity

	collision_severity	other injury	pain	property damage only	severe injury
lighting					
Unknown		580.0	1236.0	3408.0	74.0
dark with no street lights		10692.0	12085.0	48601.0	3245.0
dark with street lights		23710.0	42868.0	123434.0	5822.0
dark with street lights not functioning		226.0	386.0	1095.0	61.0
daylight		77297.0	161988.0	373991.0	12037.0
dusk or dawn		4149.0	7881.0	19716.0	896.0

Percentage of collisions for given lighting Condition and Collision Severity

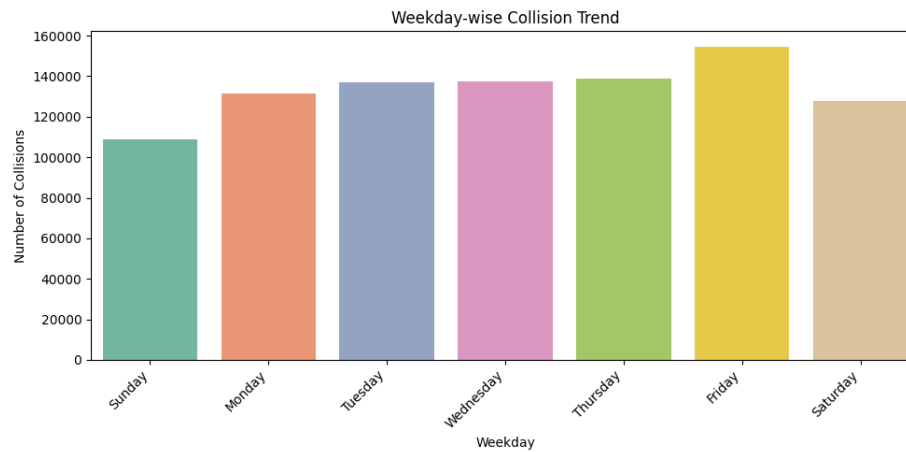
	collision_severity	other injury	pain	property damage only	severe injury
lighting					
Unknown		10.95	23.33	64.33	1.40
dark with no street lights		14.33	16.19	65.13	4.35
dark with street lights		12.11	21.89	63.03	2.97
dark with street lights not functioning		12.78	21.83	61.93	3.45
daylight		12.36	25.91	59.81	1.92
dusk or dawn		12.71	24.14	60.40	2.74



- **Property-damage-only** collisions constitute most crashes across all lighting conditions.
- **Low-visibility conditions**—such as dusk/dawn and dark environments—exhibit a higher proportion of injury-related collisions compared to daylight.
- **Severe injury collisions** remain relatively rare across all lighting types

3.1.8 Weekday wise Collision Trends –

weekday_num	weekday	count	percentage
1	Sunday	109060	11.66
2	Monday	131280	14.03
3	Tuesday	136931	14.64
4	Wednesday	137234	14.67
5	Thursday	138648	14.82
6	Friday	154483	16.51
7	Saturday	127842	13.67



- Collisions are highest on Friday
- Mid-week (Tuesday, Wednesday, and Thursday) are very close to each other and only slightly below Friday.
- Monday is moderately higher than weekend days.
- Weekend days are lower than weekdays,
 - Sunday being the lowest
 - Saturday slightly higher than Sunday, but still lower than Monday–Friday.

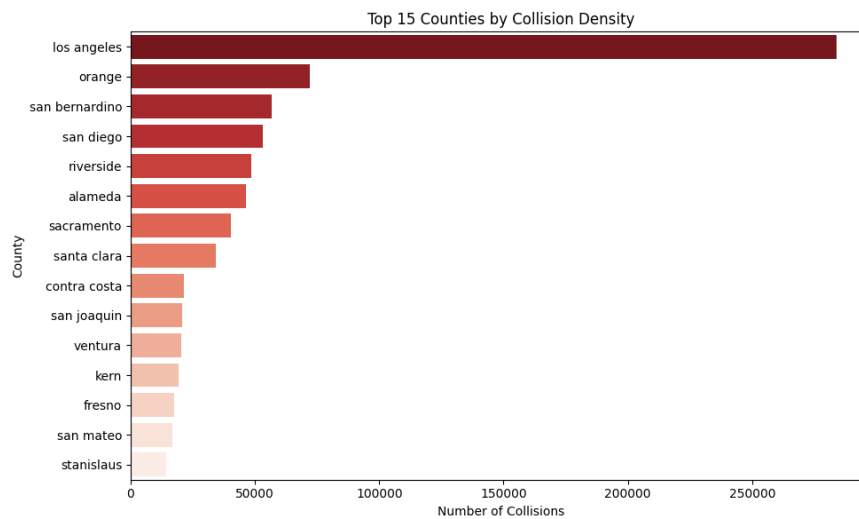
Weekdays have higher exposure to collision as travel is more due to School/Work Travel

Friday has more collisions as people are moving for weekend plans, or fatigue after a work week, heavier end of week traffic volume

Weekend drop in Collisions is due to less traffic

3.1.9 Spatial Distribution of Collision

county_location	collision_count
los angeles	284027
orange	72025
san bernardino	56725
san diego	53084
riverside	48667
alameda	46502
sacramento	40489
santa clara	34102
contra costa	21458
san joaquin	20612
ventura	20227
kern	19345
fresno	17435
san mateo	16698
stanislaus	14290



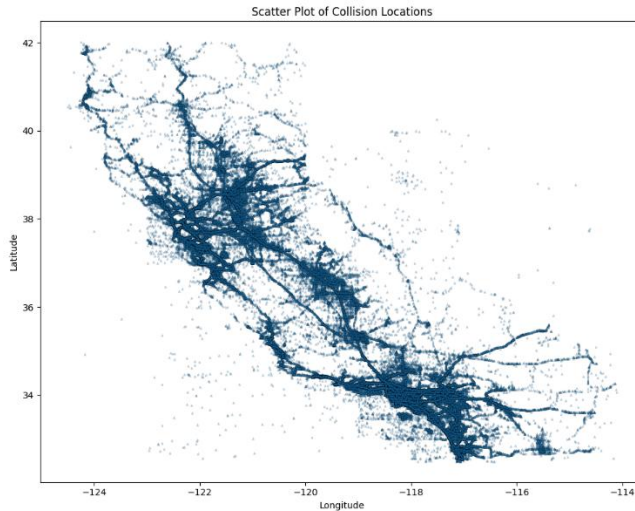
Los Angeles County has the highest collision count and is **the Collision hotspot**

The gap between Los Angeles and the second-ranked county (Orange) is very large, indicating strong spatial concentration of collisions. This might be due to high population density, heavy daily traffic volumes, and urban complexity.

A small number of large, urban counties account for most collisions.

3.1.10 Collision Analysis by Geography

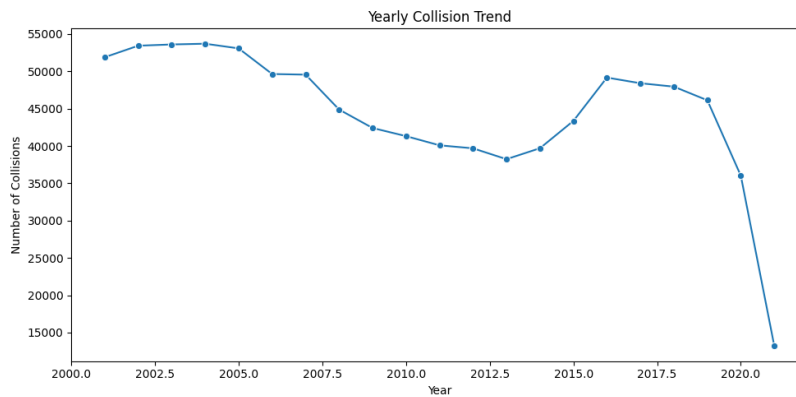
Generate a scatter plot to analyze collision locations geographically using latitudes and longitudes



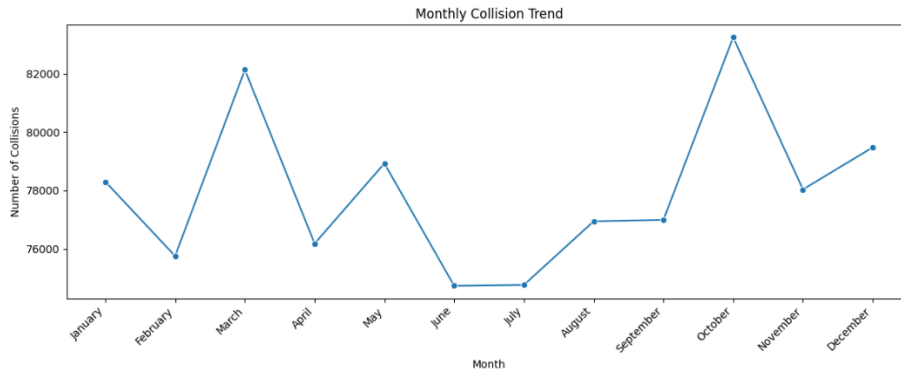
3.1.11 Collision trends over time

Extract and analyzing collision trends over time.

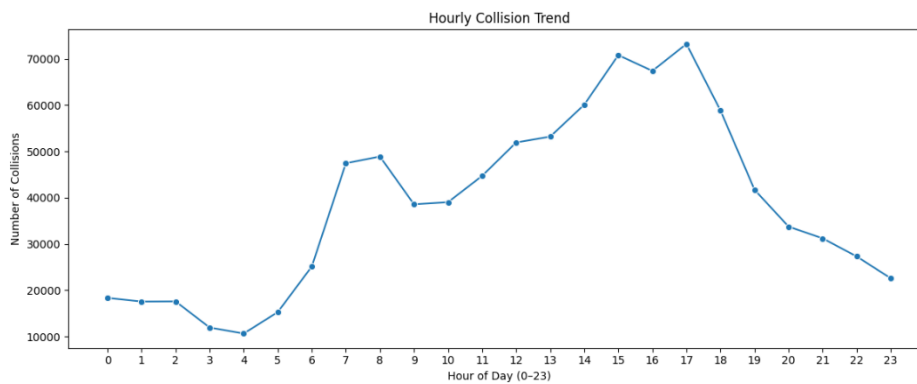
Yearly trends in Collision



Monthly Trends in Collision



Hourly Trends in Collision



4. ETL Querying

4.1 Top 5 counties

Q: Identify the top 5 counties with the highest number of collisions.

```
from pyspark.sql.functions import col, desc
```

```
top_5_counties = (  
    final_df  
        .groupBy("county_location")  
        .count()  
        .orderBy(desc("count"))  
        .limit(5)  
)
```

```
top_5_counties.show(truncate=False)
```

```
+-----+-----+  
|county_location|count |  
+-----+-----+  
|los angeles    |284027|  
|orange         |72025 |  
|san bernardino |56725 |  
|san diego      |53084 |  
|riverside      |48667 |  
+-----+-----+
```


4.2 Month with highest collisions –

Q. Identify the month with the highest number of collisions.

```
from pyspark.sql.functions import month, date_format, col, desc

month_with_max_collisions = (
    final_df
    .withColumn("month_num", month("collision_date"))
    .withColumn("month", date_format("collision_date", "MMMM"))
    .groupBy("month_num", "month")
    .count()
    .orderBy(desc("count"))
    .limit(1)
)

month_with_max_collisions.show(truncate=False)
```

```
.. +-----+-----+-----+
   |month_num|month  |count|
   +-----+-----+-----+
   |10       |October|83244|
   +-----+-----+-----+
```

4.3 Weather Conditions with Highest Collisions.

Q. Determine the most common weather conditions during collisions.

```
most_common_weather = (
    final_df
    .filter(col("weather_1").isNotNull())
    .groupBy("weather_1")
    .count()
    .orderBy(desc("count"))
    .limit(1)
)

most_common_weather.show(truncate=False)
```

```
+-----+-----+
|weather_1|count |
+-----+-----+
|clear    |769680|
+-----+-----+
```

4.4 Fatal Collisions

Q. Calculate the percentage of collisions that resulted in fatalities.

```
from pyspark.sql.functions import col, count, sum as spark_sum, when, round

fatality_percentage = (
    final_df
    .agg(
        count("*").alias("total_collisions"),
        spark_sum(
            when(col("killed_victims") > 0, 1).otherwise(0)
        ).alias("fatal_collisions")
    )
    .withColumn(
        "fatality_percentage",
        round(
            (col("fatal_collisions") / col("total_collisions")) * 100, 2
        )
    )
)

fatality_percentage.select("fatality_percentage").show(truncate=False)
```

```
+-----+
|fatality_percentage|
+-----+
|0.0                |
+-----+
```

4.5 Dangerous Time for Collisions

Q. Find the most dangerous time of day for collisions.

```
df_time = final_df.withColumn(
    "collision_time",
    F.to_timestamp("collision_time")
)

#Extract hour of day (0-23)
df_with_hour = df_time.withColumn(
    "hour_of_day",
    F.hour(F.col("collision_time"))
)

#Group by hour and count collisions
hourly_collision_counts = (
    df_with_hour
    .groupBy("hour_of_day")
    .count()
)

#Identify the most dangerous time of day
most_dangerous_time = (
    hourly_collision_counts
    .orderBy(F.desc("count"))
    .limit(1)
)

# Step 5: Display result
most_dangerous_time.show(truncate=False)
```

```
+-----+-----+
|hour_of_day|count|
+-----+-----+
|17         |73236|
+-----+-----+
```

4.6 Road Surface Conditions

Q. Identify the top 5 road surface conditions with the highest collision frequency.

```
road_surface_counts = (  
    final_df  
    .groupBy("road_surface")  
    .count()  
    .orderBy(F.desc("count"))  
    .limit(5)  
)  
  
road_surface_counts.show(truncate=False)
```

```
+-----+-----+  
|road_surface|count |  
+-----+-----+  
|dry         |845343|  
|wet         |76820 |  
|Unknown     |8137  |  
|snowy       |4120  |  
|slippery    |1048  |  
+-----+-----+
```

4.7 Lighting Conditions

Q. Analyze lighting conditions that contribute to the highest number of collisions.

```
top_3_lighting_conditions = (  
    final_df  
    .groupBy("lighting")  
    .count()  
    .orderBy(F.desc("count"))  
    .limit(3)  
)  
  
top_3_lighting_conditions.show(truncate=False)
```

```
+-----+-----+  
|lighting           |count |  
+-----+-----+  
|daylight           |625313|  
|dark with street lights |195834|  
|dark with no street lights|74623 |  
+-----+-----+
```

5. Conclusion

5.1 Recommendations to improve road safety by identifying high-risk locations and peak accident times for infrastructure improvements.

Insights -

1. Collision counts are concentrated in specific geographic hotspots (high-density zones).
2. Peak collision times occur during 2PM - 6PM

Recommendations -

1. Infrastructure should be upgraded (road resurfacing, signage, lane markings) at high-collision locations.
2. Increase traffic monitoring and policing during peak accident hours
3. Improve junction design (roundabouts, protected turns) at locations with repeated collisions.

5.2 Suggestions to optimize traffic management by analyzing trends in collision severity, weather conditions, and lighting to improve road design and traffic signal timing.

Insights -

1. Collision severity rises under poor lighting and adverse weather.

Recommendations -

1. Reduce Speed Limits in accident prone roads
2. Keep longer time duration for amber in traffic Signal
3. Upgrade to high-performance LED street lighting in low-visibility areas.
4. Fix the Potholes if any
5. Improve road drainage
6. Introduce Message signs in high-risk zones

5.3 Propose data-driven policy changes to enhance pedestrian and cyclist safety based on collision trends involving vulnerable road users.

Insights –

Pedestrians, cyclists are impacted in: High-density urban areas, Low-lighting conditions, Peak traffic hours

Recommendations -

- Mandate protected bike lanes and raised pedestrian crossings in collision zones.
- Extend pedestrian crossing signal times, especially near schools and hospitals.
- Enforce lower speed limits in residential areas.

5.4 Identify potential high-risk zones for proactive intervention by examining geographic collision density and historical accident data.

Insights - Collision density maps display collision hotspots. More number of collisions has happened in few co ordinates.

Recommendations -

1. Increase Monitoring and Policing in high-risk zones
2. Place sensors/Cameras to monitor the speed to avoid speeding related collisions
3. Introduce message signs in high-risk zones to alert people

5.5 Assess the impact of environmental factors such as weather, road surface conditions, and lighting on accident frequency and severity.

Insights -

1. Lighting correlates with collision frequency.
2. Weather and road surface condition influence severity more than frequency.
3. Poor visibility increases serious injuries.

Recommendations -

1. Improve streetlights in high-risk zones.
2. Integrate real-time weather alerts into traffic management systems.
3. Improve road surface and apply anti-skid surfacing in high-risk zones

5.6 Develop predictive models to anticipate collision hotspots and support proactive safety measures.

1. Develop predictive models using: Location, Time of day, Weather, lighting, road surface, Historical collision density
2. Use models to: Forecast emerging hotspots, Reduce vehicle speed in possible collision hotspot, Guide infrastructure investments
3. Integrate models with city traffic control systems for real-time interventions.